

Laboratorio 3

Programmazione di Sistema | A.A. 2025–26 | 20 e 24 Aprile 2026

Nota

Per esempi e suggerimenti sui tratti della libreria standard fare riferimento a: [pretzelhammer/rust-blog - tour-of-rusts-standard-library-traits](https://pretzelhammer.com/r/rust-blog/tour-of-rusts-standard-library-traits)

1 Esercizio 1 – Tratto MySlug

1.1 Descrizione

Partendo dal codice dell'esercitazione precedente (`slugify`), definire un tratto `MySlug` che estenda `String` e `&str` con due metodi: `is_slug()` e `to_slug()`.

Il tratto deve permettere di compilare ed eseguire il seguente codice ottenendo i risultati indicati:

```
let s1 = String::from("Hello String");
let s2 = "hello-slice";

println!("{}", s1.is_slug()); // false
println!("{}", s2.is_slug()); // true

let s3: String = s1.to_slug();
let s4: String = s2.to_slug();
println!("s3:{} s4:{}", s3, s4); // s3:hello-string s4:hello-slice
```

1.2 Sviluppo

Passo 1 - Funzione `is_slug`

Aggiungere nel modulo `slugify` una funzione `is_slug(&str) -> bool` che verifichi se una stringa è già uno slug valido.

Passo 2 - Implementazione esplicita per `String` e `&str`

Definire il tratto `MySlug` e fornire due implementazioni separate per `String` e `&str`

Nota

In Rust `&str` è considerabile un tipo a sé stante: è possibile scrivere `impl MySlug for &str` esattamente come per qualsiasi altro tipo.

Passo 3 - Implementazione generica

Come passo successivo, sostituire le due implementazioni con una singola implementazione generica. Un esempio è il seguente dove bisogna sostituire `???` con il constraint corretto:

```
impl<T> MySlug for T
where
    T: ???
```

```
{  
    ...  
}
```

Nota

T deve essere un qualsiasi tipo che può essere derefrenziato come `&str`, quindi T deve implementare il tratto `AsRef` per `str`: <https://doc.rust-lang.org/std/convert/trait.AsRef.html>

Nota

Con l'implementazione generica il tratto `MySlug` sarà automaticamente disponibile per tutti i tipi che implementano il constraint scelto, inclusi tipi definiti dall'utente.

2 Esercizio 2 – ComplexNumber

2.1 Descrizione

Implementare un tipo `ComplexNumber` che supporti le operazioni aritmetiche di base, possa essere copiato, clonato, confrontato con se stesso e con un numero reale, e possa essere usato nelle collezioni standard di Rust (`Vec`, `HashMap`, `VecDeque`).

I tratti da implementare e le funzionalità richieste sono definiti dal file di test `complex_numbers.rs` fornito a parte.

2.2 Sviluppo

Passo 1 – Creare il progetto

1. Creare un nuovo progetto: `cargo new cnumbers --lib`
2. Creare la directory `tests/` nella radice del progetto
3. Copiare il file `complex_numbers.rs` in `tests/`
4. Commentare tutti i test tranne il primo

Nota

`lib.rs` è il file di default in cui creare i moduli di un crate da esportare come libreria. Se si definisce il modulo `solution` in `lib.rs` del crate `cnumbers`, da `main.rs`, dai test o da altri crate si può usarlo con `use cnumbers::solution`. Per una introduzione all'organizzazione dei crate: https://rust-classes.com/chapter_4_3

Passo 2 – Implementare ComplexNumber

In `src/lib.rs` definire un modulo `solution` contenente la struct `ComplexNumber` e i tratti richiesti.

Procedere un test alla volta: implementare quanto necessario per far passare il test corrente, poi scomentare il successivo. Seguire l'ordine dei test perché alcuni tratti dipendono da quelli precedenti.

```
pub mod solution {
    pub struct ComplexNumber {
        // da definire
    }

    // implementare i tratti richiesti dai test
}
```

Per eseguire i test del modulo da terminale:

```
cargo test --package cnumbers --test complex_numbers
```

3 Esercizio 3 – Buffer Circolare

3.1 Descrizione

Un buffer circolare è una struttura di dimensione fissa che permette di inserire elementi in coda ed estrarli dalla testa. Può essere visto come un cerchio con due indici: `head` (lettura) e `tail` (scrittura).

Quando è pieno una `write` restituisce errore; quando è vuoto una `read` restituisce `None`.

Fisicamente è realizzato con un array preallocato: quando si legge si restituisce il valore puntato da `head` e si avanza `head` di 1; quando si scrive si inserisce nella cella puntata da `tail` e si avanza `tail` di 1. Quando `head` o `tail` raggiungono la fine dell'array tornano a zero.

Nota

Quando `tail` raggiunge `head` il buffer è pieno. Quando `head` raggiunge `tail` il buffer è vuoto. In certi momenti `tail` può essere minore di `head`: i valori nel buffer non sono contigui (una parte in fondo all'array, una parte in cima).

3.2 Sviluppo

Passo 1 – Struttura generica

Rendere generica la struct e implementare i metodi base:

```
pub struct CircularBuffer<T> { /* ... */ }

impl<T> CircularBuffer<T> {
    pub fn new(capacity: usize) -> Self { todo!() }
    pub fn write(&mut self, item: T) -> Result<(), /* errore */> { todo!() }
    pub fn read(&mut self) -> Option<T> { todo!() }
    pub fn clear(&mut self) { todo!() }
    pub fn size(&self) -> usize { todo!() }
    // Scrive forzando la sovrascrittura dell'elemento piu' vecchio
    pub fn overwrite(&mut self, item: T) { todo!() }
    // Riorganizza il buffer rendendolo contiguo in memoria
    pub fn make_contiguous(&mut self) { todo!() }
}
```

Nota

Quando `tail < head` (`tail` ha raggiunto la fine ed è ritornato a zero) i valori nel buffer sono spezzati in due segmenti separati, una parte all'inizio, una parte alla fine dell'array, con lo spazio vuoto in mezzo. Non è quindi contiguo e `make_contiguous()` riorganizza il buffer, copiando in cima all'array tutti gli elementi mantenendo l'ordine di lettura, rendendolo così di nuovo contiguo.

Passo 2 – Scrivere i test

Scrivere almeno i seguenti test:

- `test_write_size` – inserire un elemento e controllare la dimensione
- `test_write_read` – inserire un elemento, leggerlo e verificare che coincida
- `test_sequence` – inserire e leggere n elementi in ordine

- `test_wrap_around` – verificare che `head` e `tail` tornino a zero correttamente
- `test_read_empty` – leggere da buffer vuoto restituisce `None`
- `test_write_full` – scrivere su buffer pieno restituisce errore
- `test_overwrite` – `overwrite` su buffer pieno sovrascrive l'elemento più vecchio
- `test_make_contiguous` – verificare posizione di `head` e `tail` dopo la chiamata

Passo 3 - Trattati aggiuntivi

Implementare i seguenti tratti:

- `Index` e `IndexMut`: `buf[0]` legge o modifica l'elemento in testa, `buf[1]` il successivo, e così via. L'indice è relativo a `head`, non all'offset reale nell'array. In caso di indice fuori bounds: `panic!`
- `Deref`: dereferenzia il buffer come `&[T]`, con inizio e fine corrispondenti a `head` e `tail`. Se il buffer non è contiguo: `panic!`
- `TryDeref`: come `Deref` ma restituisce un `Result` invece di andare in `panic` se il buffer non è contiguo.

Domanda

`Deref` restituisce `&[T]` tramite una funzione che riceve `&self`. Perché questo impedisce di chiamare `make_contiguous()` internamente? E se anche fosse possibile, perché il problema non si porrebbe comunque a runtime? Provare a scrivere codice che modifichi il buffer mentre si tiene un riferimento allo slice e osservare come reagisce il compilatore.

Domanda

Un buffer circolare generico ospita tipi omogenei. Quale meccanismo di Rust permette di ospitare tipi eterogenei senza modificare l'implementazione? Quali sono le limitazioni e come varia l'occupazione di memoria?

Bonus (facoltativo)

Provare il buffer generico con il tipo `ComplexNumber` dell'esercizio precedente.